Week 1: Introduction to Functions in Python

Introduction to Functions in Python

- Imagine writing the same block of code many times.
- Functions help us **reduce repetition**.
- Real-world analogy: a **calculator button** (like "Add" or "Multiply") is like a function—it does one job whenever you call it.

What is a Function?

- A **function** is a block of reusable code that performs a specific task.
- Functions make programs:
    - Easier to read
    - Easier to maintain
    - Reusable

Python Function Syntax:

```
def function_name(parameters):
    # block of code
    return result
```

- def → keyword to define a function
- function_name → name of the function
- parameters → values the function can receive
- return → sends back a result

Example

```
def greet():
    print("Hello, welcome to Python class!")
```

To **call** the function:

greet()

Why Use Functions?

- Saves time
- Avoids repeating code
- Organizes your work
- Makes collaboration easier

Real-Life Examples of Functions

- ATMs: deposit(), withdraw(), check_balance()
- School: calculate_average(), assign_grade()
- Online apps: login(), logout(), register_user()

Coding Exercises (Practice)

Exercise 1: Create a function say_hello() that prints *"Hello Students!"*.

Exercise 2: Write a function welcome_message(name) that prints:
"Welcome, <student_name>, to SS2 Python class!"

Exercise 3: Define a function add_numbers() that adds **10 + 20** and prints the result.

Assessment / Quiz (Check Understanding)

1. **What is a function in Python?**
2. **What keyword is used to define a function?**
3. **What is the difference between defining a function and calling it?**
4. **Write a small function school_motto() that prints your school's motto.**

Real-World Project (Week 1)

Project: Student Greeting System

Problem: A school wants a program that greets students when they enter the lab.

Task:

- Write a function greet_student(name) that accepts a student's name and prints a welcome message.

Sample Code:

```
def greet_student(name):
    print(f"Welcome {name}, to SS2 Python class. We are happy to have you!")
# Calling the function for different students
greet_student("Elisha")
greet_student("Mary")
greet_student("David")
```

Expected Output:

Welcome Elisha, to SS2 Python class. We are happy to have you!

Welcome Mary, to SS2 Python class. We are happy to have you!

Welcome David, to SS2 Python class. We are happy to have you!

Week 2: Function Parameters & Arguments

"Function Parameters & Arguments in Python"

Recap (From Last Week)

- Functions help us reuse code.
- We learned how to define and call simple functions.
- Example:

```
def greet():
  print("Hello, Students!")
```

Why Parameters?

- Functions become powerful when we pass **data** into them.
- Example: Instead of greeting the same way every time, pass a **name**.

Parameters vs. Arguments

- **Parameter** → a variable inside the function definition.
- **Argument** → the actual value you pass when calling the function.

```
def greet_student(name):   # name is the parameter
  print("Welcome", name)
greet_student("Elisha")    # "Elisha" is the argument
```

Multiple Parameters

```
def student_info(name, age, grade):
  print(f"Name: {name}, Age: {age}, Grade: {grade}")


student_info("Mary", 15, "SS2")
```

Default Parameters

- You can give a default value if no argument is passed.

```
def greet(name="Student"):
    print("Hello", name)

greet()       # Output: Hello Student
greet("John")  # Output: Hello John
```

Types of Arguments

- Positional Arguments (order matters).
- Keyword Arguments (specify explicitly).

```
def student_info(name, age):
    print(f"{name} is {age} years old.")

student_info("David", 16)        # Positional
student_info(age=16, name="David") # Keyword
```

Real-Life Examples of Parameters

- ATM → withdraw(amount), deposit(amount).
- Online shop → add_to_cart(item, quantity).
- School → record_score(student_name, subject, marks).

Coding Exercises (Practice)

Exercise 1: Create a function square(number) that returns the square of a number.

Exercise 2: Write a function student_details(name, age, class_name) that prints student information.

Exercise 3: Create a function calculate_area(length, width=5) that calculates the area of a rectangle. Try calling it with one and with two arguments.

Assessment / Quiz (Check Understanding)

1. What is the difference between a **parameter** and an **argument**?
2. Write a function greet_teacher(name) that prints "Good morning, <name>".
3. Create a function with **default arguments** for subject = "Maths" and print student details.
4. What is the advantage of using keyword arguments?

Real-World Project (Week 2)

Project: Student Information System Function

Problem: A school database needs a simple function to record and display student details.

Task:

- Create a function student_profile(name, age, class_name, subject="Maths").
- When called, it should print all student details in a structured format.

Sample Code:

```
def student_profile(name, age, class_name, subject="Maths"):
    print("---- Student Profile ----")
    print(f"Name: {name}")
    print(f"Age: {age}")
    print(f"Class: {class_name}")
    print(f"Subject: {subject}")
    print("------------------------")

# Calling function
student_profile("Mary", 15, "SS2A")
student_profile("Elisha", 16, "SS2B", "English")
```

Expected Output:

```
---- Student Profile ----
```

Name: Mary

Age: 15

Class: SS2A

Subject: Maths

---- Student Profile ----

Name: Elisha

Age: 16

Class: SS2B

Subject: English

Week 3: Return Values & Multiple Returns in Functions

Return Values & Multiple Returns in Functions

Slide 2 – Recap (From Last Week)

- Functions can take **parameters** and **arguments**.
- We built a **Student Information System function**.
- Today: learn how to make functions **send results back**.

Slide 3 – What is return in Python?

- Functions don't always just print; they can **return values**.
- return sends data back to where the function was called.

```
def add(a, b):
   return a + b


result = add(5, 3)
print(result)   # 8
```

Slide 4 – Why use return instead of print?

- print() → only shows output on the screen.
- return → stores output for later use in calculations.

```
def square(n):
   return n * n


x = square(4)
y = square(5)
print(x + y)   # 16 + 25 = 41
```

Slide 5 – Returning Multiple Values

- A function can return **more than one value** using tuples.

```
def student_scores(math, eng, sci):
```

```
    total = math + eng + sci
    average = total / 3
    return total, average

t, avg = student_scores(70, 80, 90)
print("Total:", t, "Average:", avg)
```

Slide 6 – Real-Life Examples of Return Values

- Calculator → returns the sum/product/etc.
- ATM → returns updated account balance.
- Exam system → returns total score and grade.

Coding Exercises (Practice)

Exercise 1: Write a function multiply(a, b) that returns the product of two numbers.

Exercise 2: Write a function area_perimeter(length, width) that returns both the area and perimeter of a rectangle.

Exercise 3: Create a function student_result(name, score) that returns both the student's name and whether they passed (score ≥ 50).

Assessment / Quiz (Check Understanding)

1. What is the difference between print() and return?
2. Can a function return more than one value? How?
3. Write a function divide(a, b) that returns the quotient and remainder.
4. What would happen if a function has a print() but no return?

Real-World Project (Week 3)

Project: Student Marks Analyzer

Problem: A teacher wants a program that can **calculate a student's total marks, average, and grade** using a function.

Task:

- Create a function analyze_scores(name, math, eng, sci)
- It should return:
    - **Student Name**
    - **Total Marks**
    - **Average**
    - **Grade (A-F)**

```
def analyze_scores(name, math, eng, sci):
    total = math + eng + sci
    average = total / 3

    # Decide grade
    if average >= 70:
        grade = "A"
    elif average >= 60:
        grade = "B"
    elif average >= 50:
        grade = "C"
    elif average >= 40:
        grade = "D"
    else:
        grade = "F"

    return name, total, average, grade

# Calling the function
```

```
student = analyze_scores("Elisha", 75, 68, 80)
print("Name:", student[0])
print("Total:", student[1])
print("Average:", student[2])
print("Grade:", student[3])
```

Expected Output:

Name: Elisha

Total: 223

Average: 74.33

Grade: A

Why this matters (Real-Life Application):

- Similar to **school grading software**.
- Prepares students for building **result management systems**.

Week 4: Functions with Conditional Logic

Functions with Conditional Logic in Python

Recap (From Last Week)

- We learned how to **return values** from functions.
- We even returned **multiple values** (total, average, grade).
- Today: making functions **smarter** using decisions.

Why Use Conditional Logic in Functions?

- Functions should not always give the same output.
- With **if, elif, else**, functions can make decisions.
- Example: Checking if a student **passed or failed**.

Syntax Reminder

```
def check_pass(score):
    if score >= 50:
        return "Pass"
    else:
        return "Fail"

print(check_pass(65))   # Pass
print(check_pass(40))   # Fail
```

Using Multiple Conditions

```
def grade_student(score):
    if score >= 70:
        return "A"
    elif score >= 60:
        return "B"
```

```
    elif score >= 50:
        return "C"
    else:
        return "F"
```

Combining Logic with Parameters

- Functions can use **arguments + logic** for smarter results.

```
def check_age(name, age):
    if age >= 18:
        return f"{name} is eligible to vote."
    else:
        return f"{name} is too young to vote."
```

Real-Life Examples

- ATM → if balance < amount → deny withdrawal.
- Exams → if score < 40 → fail.
- Online shops → if stock = 0 → "Out of stock."

Coding Exercises (Practice)

Exercise 1: Write a function is_even(number) that returns whether a number is even or odd.

Exercise 2: Create a function student_grade(score) that returns A, B, C, D, or F.

Exercise 3: Define a function login(username, password) that checks if the username = "admin" and password = "1234". If correct → return "Login Successful," else → "Access Denied."

Assessment / Quiz (Check Understanding)

1. Why do we use **if-else** inside functions?
2. Write a function check_temperature(temp) that prints "Hot" if temp > 30, else "Cool".

3. What is the difference between **if** and **elif**?
4. True/False: Every function must have at least one else.

Real-World Project (Week 4)

Project: Pass/Fail & Grading System Function

Problem: A teacher wants to know whether students **passed or failed**, and also their **grade**.

Task:

- Create a function student_report(name, score) that:
    - Prints the student's name.
    - Checks pass/fail (≥ 40 = Pass, otherwise Fail).
    - Assigns grade (A = 70+, B = 60–69, C = 50–59, D = 40–49, F = <40).

Sample Code:

```
def student_report(name, score):
    print("---- Student Report ----")
    print("Name:", name)
    print("Score:", score)

    # Pass/Fail Check
    if score >= 40:
        status = "Pass"
    else:
        status = "Fail"

    # Grade Check
    if score >= 70:
        grade = "A"
    elif score >= 60:
```

```
        grade = "B"
    elif score >= 50:
        grade = "C"
    elif score >= 40:
        grade = "D"
    else:
        grade = "F"

    print("Status:", status)
    print("Grade:", grade)
    print("----------------------")

# Example calls
student_report("Elisha", 72)
student_report("Mary", 35)
```

Expected Output:

```
---- Student Report ----
Name: Elisha
Score: 72
Status: Pass
Grade: A
----------------------

---- Student Report ----
Name: Mary
Score: 35
Status: Fail
```

Grade: F

Why this matters (Real-Life Application):

- Very similar to **school report systems**.
- Logic-based decisions are used in **banking, e-learning, and mobile apps**.

Week 5: Functions with Loops

Using Loops inside Functions in Python

- We used **if–elif–else** in functions to check pass/fail and assign grades.
- Today: make functions more powerful using **loops**.

Why Loops in Functions?

- Loops allow **repeating tasks** inside functions.
- Examples:
    - Generating a **multiplication table**.
    - Printing **student names from a list**.
    - Summing multiple numbers.

For Loop in Functions

```
def print_numbers():
    for i in range(1, 6):
        print(i)


print_numbers()
```

While Loop in Functions

```
def countdown(n):
    while n > 0:
        print(n)
        n -= 1
    print("Time's up!")


countdown(5)
```

Combining Loops & Parameters

```
def multiplication_table(number):
    for i in range(1, 13):
```

```
    print(f"{number} x {i} = {number * i}")


multiplication_table(5)
```

Real-Life Uses

- ATM: loop through transactions.
- Exams: loop through scores of students.
- School: generate attendance list.

Coding Exercises (Practice)

Exercise 1: Write a function print_even_numbers(n) that prints all even numbers from 1 to n.

Exercise 2: Create a function sum_of_list(numbers) that takes a list of numbers and returns their sum using a loop.

Exercise 3: Define a function word_repeater(word, times) that prints a word repeatedly based on the given number.

Assessment / Quiz (Check Understanding)

1. Why are loops useful inside functions?
2. Write a function that prints the multiplication table of 7.
3. What is the difference between for and while loops?
4. True/False: A function can contain only one loop.

Real-World Project (Week 5)

Project: Multiplication Table Generator

Problem: Students need a program that generates multiplication tables for any number they choose.

Task:

- Create a function generate_table(number) that prints the multiplication table of number from 1 to 12.
- Extend it to handle **multiple numbers** using a loop.

Sample Code:

```
def generate_table(number):
    print(f"--- Multiplication Table for {number} ---")
    for i in range(1, 13):
        print(f"{number} x {i} = {number * i}")
    print("--------------------------")

# Generate table for a single number
generate_table(5)

# Generate tables for multiple numbers
for n in range(2, 6):
    generate_table(n)
```

Expected Output (Partial):

```
--- Multiplication Table for 5 ---
5 x 1 = 5
5 x 2 = 10
...
5 x 12 = 60
--------------------------

--- Multiplication Table for 2 ---
2 x 1 = 2
2 x 2 = 4
...
2 x 12 = 24
--------------------------
```

Why this matters (Real-Life Application):

- Similar to **math apps** that generate tables.
- Useful for students practicing multiplication.
- Foundation for building **quiz and e-learning tools**.

Week 6: Functions and Decision Making

Lesson Focus

- Using functions with if, elif, and else statements.
- Making decisions inside functions.
- Returning different outputs depending on conditions.

Lesson Outline

1. **Revision of Functions** – parameters & return values.
2. **Decision Making in Functions**
    - Functions can contain conditional statements.
    - Example:
    - def check_pass(score):
    - if score >= 50:
    - return "Pass"
    - else:
    - return "Fail"
    -
    - print(check_pass(65)) # Output: Pass
3. **Multiple Conditions in Functions**
4. def grade(score):
5. if score >= 70:
6. return "A"
7. elif score >= 60:
8. return "B"
9. elif score >= 50:
10. return "C"

```
    else:
        return "F"

print(grade(68))  # Output: B
```

15. **Real-World Analogy** – Like a teacher deciding grades for students, a function can "decide" outcomes.

Coding Exercises

1. Write a function check_temperature(temp) that:
    - Returns "Hot" if temp ≥ 30
    - Returns "Warm" if temp ≥ 20 but < 30
    - Returns "Cold" if temp < 20
2. Write a function student_status(score) that:
    - Returns "Excellent" if score ≥ 80
    - Returns "Good" if 60–79
    - Returns "Average" if 50–59
    - Returns "Poor" if < 50

Assessment (Mini Quiz)

- What is the difference between a function returning "Pass" or "Fail" vs printing it directly?
- Write a function check_age(age) that:
    - Returns "Minor" if age < 18
    - Returns "Adult" if 18–60
    - Returns "Senior Citizen" if > 60

✅ **Real-World Project (Week 6)**

Project: Student Result Grading System

- Write a function result_report(name, score) that:

- Takes in student name and score.
- Uses decision-making to determine grade.
- Returns a sentence like:
  "John scored 78, which is Grade B – Good performance."

Extension: Make the function check multiple students (using a list of scores).

Week 7: Functions and Loops

Lesson Focus

- Using **for** and **while loops** inside functions.
- Returning results after looping.
- Understanding why loops and functions work well together.

Lesson Outline

1. **Revision of Loops**
    - for loop → used for a known number of iterations.
    - while loop → used when condition is not fixed.

Functions with Loops

```
def print_numbers(n):
  for i in range(1, n+1):
    print(i)


print_numbers(5)
# Output: 1 2 3 4 5
```

Returning Values After Looping

```
def squares(n):
        result = []
  for i in range(1, n+1):
    result.append(i**2)
  return result


print(squares(5))  # Output: [1, 4, 9, 16, 25]
```

Functions with While Loop

```
def countdown(n):
  while n > 0:
    print(n)
    n -= 1
  print("Time's up!")


    countdown(5)
```

Coding Exercises

1. Write a function sum_numbers(n) that:
    - Uses a loop to sum numbers from 1 to n.
    - Returns the total.
2. Write a function multiplication_table(n) that:
    - Prints the multiplication table of n (from 1 to 12).

Assessment (Mini Quiz)

- Which is better for checking attendance of students: for loop or while loop? Why?
- Write a function even_numbers(n) that returns a list of even numbers from 1 to n.

Real-World Project (Week 7)

Project: School Attendance Tracker

- Write a function attendance(names) that:
    - Takes a list of student names.
    - Uses a loop to print:
      "Present: [Student Name]" for each student.

Extension: Count how many students are present and return the number.

Week 8: Functions with Lists & Dictionaries

Lesson Focus

- Passing lists and dictionaries into functions.

- Returning lists/dictionaries from functions.
- Using loops + functions + lists/dictionaries to solve real problems.

Lesson Outline

1. **Functions with Lists**
2. def average_score(scores):
3. total = sum(scores)
4. return total / len(scores)
5.

```
print(average_score([60, 75, 80, 90]))  # Output: 76.25
```

Functions Returning Lists

```
def filter_pass(scores):
    passed = []
    for score in scores:
        if score >= 50:
            passed.append(score)
    return passed

print(filter_pass([30, 45, 55, 70, 85]))  # Output: [55, 70, 85]
```

Functions with Dictionaries

```
def get_student_grade(student):
    return f"{student['name']} scored {student['score']}"
```

```
student1 = {"name": "Grace", "score": 78}
print(get_student_grade(student1))
# Output: Grace scored 78
```

Functions Returning Dictionaries

```
def student_report(name, scores):
    average = sum(scores) / len(scores)
    return {"name": name, "average": average, "scores": scores}

print(student_report("James", [70, 80, 90]))
# Output: {'name': 'James', 'average': 80.0, 'scores': [70, 80, 90]}
```

Coding Exercises

1. Write a function highest_score(scores) that returns the highest score from a list.
2. Write a function student_info(student) that prints name, age, and class from a dictionary.
3. Write a function get_failed(scores) that returns all scores below 50 from a list.

Assessment (Mini Quiz)

- What is the difference between passing a list and a dictionary to a function?
- Write a function calculate_total(students) that accepts a list of dictionaries with names and scores, and returns the total score of all students.

Real-World Project (Week 8)

Project: Student Report System

- Write a function class_report(students) that:

- Accepts a list of student dictionaries. Example:
- students = [
- {"name": "John", "scores": [70, 80, 65]},
- {"name": "Mary", "scores": [90, 85, 95]},
- {"name": "Peter", "scores": [40, 55, 60]}
-]
- For each student:
  - Calculate the average.
  - Print: "Mary's average is 90.0 → Excellent".
- Return a dictionary with all students and their average.

Extension: Count how many students passed and how many failed.

Week 9: Functions with String Manipulation

Lesson Focus

- Passing strings into functions.
- Using string methods inside functions.
- Returning modified strings.
- Real-life applications of string manipulation (text processing, formatting reports, etc.).

Lesson Outline

1. **Functions with Strings**
2. def greet(name):
3. return f"Hello, {name}!"
4.

```
print(greet("Samuel"))  # Output: Hello, Samuel!
```

Using String Methods Inside Functions

```
def format_text(text):
   return text.strip().capitalize()

print(format_text("   hello world   "))
# Output: Hello world
```

Counting Words in a String

```
def word_count(sentence):
```

```
    words = sentence.split()
    return len(words)

print(word_count("Python is fun to learn"))  # Output: 5
```

Checking for Keywords

```
def contains_keyword(text, keyword):
    return keyword.lower() in text.lower()

print(contains_keyword("I love coding in Python", "python"))
# Output: True
```

Coding Exercises

1. Write a function reverse_string(word) that returns the reversed string.
2. Write a function count_vowels(word) that returns how many vowels are in a word.
3. Write a function is_palindrome(word) that checks if a word reads the same backward (e.g., "level").

Assessment (Mini Quiz)

- What string method would you use to:
    1. Remove spaces at the beginning and end of a string?
    2. Change all letters in a string to uppercase?
- Write a function longest_word(sentence) that returns the longest word in a sentence.

Real-World Project (Week 9)

Project: Text Analyzer for School Essays

- Write a function analyze_essay(text) that:

    - Counts the number of words.
    - Counts the number of sentences (split by ".").
    - Counts how many times the word "student" appears.
- Print a summary report like:
- Total words: 120
- Total sentences: 8
- "student" appears 5 times

Extension: Return the top 3 most common words in the essay.

Week 10: Functions with File Handling

Lesson Focus

- Using functions to read from and write to files.
- Passing file names to functions.
- Returning file data from functions.
- Applying file handling to real-world school tasks (records, reports, attendance).

Lesson Outline

Writing to a File Inside a Function

```
def save_note(filename, text):
    with open(filename, "w") as file:
        file.write(text)
    return "Note saved successfully!"

print(save_note("note.txt", "This is my first note."))
```

Reading from a File Inside a Function

```
def read_note(filename):
    with open(filename, "r") as file:
        return file.read()

print(read_note("note.txt"))
```

Appending to a File

```
def add_log(filename, log):
    with open(filename, "a") as file:
        file.write(log + "\n")
```

```
    return "Log added!"


add_log("activity.txt", "Student John submitted homework.")
add_log("activity.txt", "Mary scored 85 in Maths.")
```

 Returning Data from Files

```
def get_lines(filename):
    with open(filename, "r") as file:
        return file.readlines()


print(get_lines("activity.txt"))
```

Coding Exercises

1. Write a function save_student(name, score) that writes a student's name and score into students.txt.
2. Write a function get_students() that reads all students from students.txt and prints them.
3. Write a function clear_file(filename) that empties all contents of a file.

Assessment (Mini Quiz)

- Which mode ("w", "a", "r") would you use to:
    1. Add a new line without deleting old data?
    2. Read only?
- Write a function count_lines(filename) that returns how many lines are in a file.

Real-World Project (Week 10)

Project: Student Attendance File System

- Write a function mark_attendance(filename, student_name) that:

    - Opens the file in append mode.
    - Writes: "John - Present" each time a student is marked.
- Write another function view_attendance(filename) that:
    - Reads the file.
    - Prints a formatted attendance list.

Extension: Count the number of students marked as present.

Week 11: Functions with Error Handling

Lesson Focus

- Understanding errors (syntax vs runtime errors).
- Using try...except inside functions.
- Returning safe results instead of program crashes.
- Applying error handling in real-world scenarios (calculators, file reading, student systems).

Lesson Outline

Basic Error Handling in Functions

```
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Error: Cannot divide by zero!"


print(divide(10, 2))   # Output: 5.0
print(divide(8, 0))    # Output: Error: Cannot divide by zero!
```

Handling Input Errors

```
def get_age():
    try:
        age = int(input("Enter your age: "))
        return f"You are {age} years old."
    except ValueError:
        return "Invalid input! Please enter a number."


print(get_age())
```

Handling File Errors

```
def read_file(filename):
    try:
        with open(filename, "r") as file:
            return file.read()
    except FileNotFoundError:
        return "File not found!"

print(read_file("students.txt"))
```

Multiple Exceptions

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Cannot divide by zero"
    except TypeError:
        return "Both inputs must be numbers"

print(safe_divide(10, "two"))  # Output: Both inputs must be numbers
```

Coding Exercises

1. Write a function safe_input() that asks for a number and handles invalid input.
2. Write a function open_student_file(filename) that handles missing file errors gracefully.
3. Write a function add_score(score) that only accepts integers; otherwise, return an error message.

Assessment (Mini Quiz)

- What is the difference between ValueError and TypeError?

- Write a function safe_list_access(my_list, index) that:
    - Tries to return the item at index.
    - If out of range, return "Invalid index".

Real-World Project (Week 11)

Project: Safe Grade Calculator

- Write a function calculate_grade(marks, total) that:
    - Uses try...except to handle errors.
    - Calculates percentage = (marks / total) * 100.
    - Handles:
        - Division by zero.
        - Wrong data types (e.g., text instead of numbers).
    - Returns a letter grade (A, B, C, etc.).

Extension: Store the result safely in a file, handling FileNotFoundError.

Week 12: Functions with Recursion

Lesson Focus

- Understanding recursion (a function calling itself).
- Base case vs recursive case.

- Examples: factorial, Fibonacci, sum of numbers.
- Real-world applications of recursion (searching, problem-solving).

Lesson Outline

1. **Introduction to Recursion**
    - A function that calls itself.
    - Must have a **base case** (to stop infinite calls).

```
def countdown(n):
    if n == 0:
        print("Time's up!")
    else:
        print(n)
        countdown(n - 1)

countdown(5)
```

Factorial Example

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5))  # Output: 120
```

Fibonacci Sequence

```
def fibonacci(n):
    if n <= 1:
        return n
```

```
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

for i in range(6):
    print(fibonacci(i), end=" ")
# Output: 0 1 1 2 3 5
```

Sum of Numbers (1 to n)

```
def sum_numbers(n):
    if n == 0:
        return 0
    else:
        return n + sum_numbers(n - 1)

print(sum_numbers(10))  # Output: 55
```

Coding Exercises

1. Write a recursive function reverse_string(word) that returns the reversed word.
2. Write a recursive function power(base, exp) that calculates base^exp.
3. Write a recursive function gcd(a, b) to find the greatest common divisor.

Assessment (Mini Quiz)

- What is the difference between a base case and a recursive case?
- Why is recursion risky if no base case is defined?
- Write a recursive function count_vowels(word) that counts vowels in a string.

Real-World Project (Week 12)

Project: Directory File Counter (Simulated)

- Write a recursive function count_files(structure) that:

  Accepts a nested dictionary representing folders and files. Example:

  files = {

    "Documents": {

      "school": ["maths.docx", "english.pdf"],

      "projects": ["python.py", "science.txt"]

    },

    "Pictures": ["holiday.jpg", "family.png"]

  }

  If item is a **file** → count it.

  If item is a **folder** → call the function again.

- Return the **total number of files**.

Extension: Modify the function to also return the number of folders.

Week 13: Capstone Project – Functions Integration

Lesson Focus

- Bringing together all function concepts:
    - Parameters & return values
    - Loops inside functions
    - Lists & dictionaries
    - String manipulation
    - File handling
    - Error handling
    - Recursion (optional advanced part)
- Building a **real-world project** that simulates real school/work scenarios.

Lesson Outline

1. **Review of Function Concepts**
    - Define and call functions.
    - Return values.
    - Functions with data structures.
    - Error handling in functions.
    - Real-world use cases.
2. **Why Capstone?**
    - To prove mastery of concepts.
    - To connect coding with practical student problems.
    - To simulate real software development tasks.

Capstone Project

Project Title: School Management System (Mini Version)

Project Description:

Students will build a Python program using functions that can:

1. **Register Students**
    - Save student name, age, and class into a file.
2. **Record Scores**
    - Input scores for subjects.
    - Store them in a dictionary.
3. **Generate Report Card**
    - Calculate average.
    - Assign grades (A, B, C, D, F).
    - Handle errors (e.g., wrong inputs).
4. **Attendance Tracker**
    - Mark students as present or absent.
    - Store attendance in a file.
5. **Search Feature**
    - Search for a student's record by name.

Code Structure (Starter Guide)

```
# 1. Register Student
def register_student(filename, name, age, clas):
   with open(filename, "a") as file:
      file.write(f"{name},{age},{clas}\n")
   return f"Student {name} registered successfully!"


# 2. Record Scores
def record_scores(name, scores):
```

```
    return {"name": name, "scores": scores, "average": sum(scores)/len(scores)}


# 3. Grade Generator
def get_grade(average):
    if average >= 70:
        return "A"
    elif average >= 60:
        return "B"
    elif average >= 50:
        return "C"
    elif average >= 40:
        return "D"
    else:
        return "F"


# 4. Report Card
def report_card(student):
    avg = student["average"]
    grade = get_grade(avg)
    return f"{student['name']} Average: {avg:.2f} Grade: {grade}"


# Example Run
student = record_scores("John", [70, 80, 90])
print(report_card(student))
```

Assessment (Capstone Evaluation Criteria)

1. **Correct Use of Functions** (30%)

2. **Integration of Lists, Dicts, Loops, Error Handling** (30%)
3. **Working Real-World Features (Registration, Report, Attendance, Search)** (30%)
4. **Creativity & Extension Features** (10%)

Extensions for Advanced Students

- Add recursion for searching students in nested data.
- Add a login system for teachers.
- Export report card to a .txt file.